Configuration Guide
Message Transformation Guidelines - JSONDataTypeConvertor
August 2026
English

Message Transformation Guidelines – JSONDataTypeConvertor 2.0.0

Content

1 Introduction

In SAP Cloud Integration there is an integration object called 'XML to JSON Converter' that can be used to convert an XML payload to JSON format. However, this conversion transforms all the field into String type, not being possible to configure any other types, except arrays. In SAP PI/PO, these conversions are done using the adapter module 'XMLtoJSONConverter', where it is possible to configure the expected data type for each field.

This document presents the groovy script 'JSONDataTypeConvertor', an alternative solution created to convert JSON properties in a payload from String to Integer, Decimal, or Boolean or vise-versa.

This groovy script is not a replacement for the integration object 'XML to JSON Converter', it is meant to be used after the XML to JSON conversion takes place to enforce the expected data types.

The document also describes in detail the required and available properties/configurations and how to use them, followed by an example for each scenario.

1.1 Properties

Property	Values	Default	Required	Description
`json.dataTypeConvertor.wrongType`	error / ignore	error	No	Whether to throw an exception or silently skip when a value cannot be converted to the target type.
`json.dataTypeConvertor.integer`	semicolon-separated paths	—	No	Converts the specified fields from String to Integer. If the value has decimals (e.g. "2.5"), they are truncated (result: 2).
`json.dataTypeConvertor.decimal`	semicolon-separated paths	—	No	Converts the specified fields from String to Decimal.
`json.dataTypeConvertor.boolean`	semicolon-separated paths	—	No	Converts the specified fields from String to Boolean. Value must be true or false (case-insensitive).
`json.dataTypeConvertor.string`	semicolon-separated paths	—	No	Converts the specified fields from any type to String.
`json.dataTypeConvertor.null`	semicolon-separated paths	—	No	Converts fields in the null format produced by the standard XML to JSON converter to a proper JSON null. (e.g. "foo": {"@nil": "true"} → "foo": null)
`json.dataTypeConvertor.dynamic`	semicolon-separated paths	—	No	Dynamically converts the specified fields based on their value. Numeric strings convert to Integer or Decimal, true/false strings convert to Boolean, and the {@nil: true} pattern converts to null. String values with leading zeros (e.g. "0042") or values exceeding the digitMaxRange are kept as strings. Non-string values are left unchanged.
`json.dataTypeConvertor.formatAll`	dynamic / string	—	No	Applies a conversion to all fields without listing them individually. dynamic applies the same logic as the dynamic property to every field. string converts every leaf value to String. Explicit typed properties take precedence over formatAll for the fields they configure. If not set or set to any other value, no format-all conversion is applied.
`json.dataTypeConvertor.digitMaxRange`	integer / long	long	No	Sets the maximum numeric range for dynamic conversion. integer limits to 32-bit signed (max

				2,147,483,647 — 10 digits). long limits to 64-bit signed (max 9,223,372,036,854,775,807 — 19 digits). Numeric strings exceeding the range are kept as strings. Applies to dynamic property and formatAll=dynamic.
json.dataTypeConvertor.prettyPrint	true / false	false	No	If true, the output JSON is formatted with indentation.
json.dataTypeConvertor.escapeInvalidStart	true / false	false	No	If true, field names starting with an XML-invalid character are prefixed with the escapeChar to allow correct JSON to XML conversion.
json.dataTypeConvertor.mangleInvalidChars	true / false	false	No	If true, XML-invalid characters within field names are replaced with their Unicode escape using the pattern {escapeChar}u{hex}{escapeChar} (e.g. & → _u0026_).
json.dataTypeConvertor.escapeChar	any string	_	Mandatory if escapeInvalidStart or mangleInvalidChars is true	The character used as prefix/wrapper when escaping or mangling field names.

Multiple JSON fields can be set in each property, using a semicolon (;) as the field delimiter. To convert fields from nested objects, the path to the field must be added like an XPATH, using a slash (/) as the path delimiter.

Ex Payload:

```
{
  "college": {
    "collegeDetails": {
      "contactEmail": "info@abcuniversity.edu",
      "established": "1965",
      "location": "New York, USA",
      "name": "ABC University",
      "number": "10",
      "bool": "true"
    }
}
```

json.dataTypeConvertor.integer=college/collegeDetails/established;college/collegeDetails/number

json.dataTypeConvertor.boolean=college/collegeDetails/bool

1.2 Examples

1.2.1 Full dynamic conversion

1. Input Properties:

   json.dataTypeConvertor.dynamic=users/id;users/isActive;users/accountBalance;users/profile/age;users/profile/height;users/profile/preferences/email;users/foo

2. Input Payload:

```
{
  "users": [
```

```
    {
      "accountBalance": "2489.75",
      "email": "alice.johnson@example.com",
      "id": "1023",
      "isActive": "true",
      "name": "Alice Johnson",
       "foo": {
            "@nil": "true"
          },
      "profile": {
        "age": "34",
        "height": "5.7",
        "preferences": {
          "email": "false",
          "theme": "dark"
        }
      }
    },
    {
      "accountBalance": "172.40",
      "email": "brian.lee@example.net",
      "id": "2045",
      "isActive": "false",
      "name": "Brian Lee",
      "profile": {
        "age": "29",
        "height": "6.1",
        "preferences": {
          "email": "true",
          "theme": "light"
        }
      }
    },
    {
      "accountBalance": "9300.00",
      "email": "cynthia.gomez@example.org",
      "id": "3087",
      "isActive": "true",
      "name": "Cynthia Gomez",
      "profile": {
        "age": "41",
        "height": "5.4",
        "preferences": {
          "email": "true",
          "theme": "auto"
        }
      }
    }
  ]
}
```

3. Output Payload:


```
{
  "users": [
    {
      "accountBalance": 2489.75,
      "email": "alice.johnson@example.com",
      "id": 1023,
      "isActive": true,
      "name": "Alice Johnson",
      "foo": null,
      "profile": {
        "age": 34,
        "height": 5.7,
        "preferences": {
          "email": false,
          "theme": "dark"
        }
      }
    },
    {
      "accountBalance": 172.40,
      "email": "brian.lee@example.net",
      "id": 2045,
      "isActive": false,
      "name": "Brian Lee",
      "profile": {
        "age": 29,
        "height": 6.1,
        "preferences": {
          "email": true,
          "theme": "light"
        }
      }
    },
    {
      "accountBalance": 9300.00,
      "email": "cynthia.gomez@example.org",
      "id": 3087,
      "isActive": true,
      "name": "Cynthia Gomez",
      "profile": {
        "age": 41,
        "height": 5.4,
        "preferences": {
          "email": true,
          "theme": "auto"
        }
      }
    }
  ]
}
```

1.2.2 Integer, Decimal and Boolean conversion

1. Input Properties:
   json.dataTypeConvertor.integer= users/accountBalance;users/profile/height
   json.dataTypeConvertor.decimal= users/profile/age
   json.dataTypeConvertor.boolean= users/isActive;users/profile/preferences/email

2. Input Payload:

➔ Same as the **INPUT** of the previous sample.

3. Output Payload:


```
{
  "users": [
    {
      "accountBalance": 2489.75,
      "email": "alice.johnson@example.com",
      "id": "1023",
      "isActive": true,
      "name": "Alice Johnson",
      "profile": {
        "age": 34,
        "height": 5.7,
        "preferences": {
          "email": false,
          "theme": "dark"
        }
      }
    },
    {
      "accountBalance": 172.4,
      "email": "brian.lee@example.net",
      "id": "2045",
      "isActive": false,
      "name": "Brian Lee",
      "profile": {
        "age": 29,
        "height": 6.1,
        "preferences": {
          "email": true,
          "theme": "light"
        }
      }
    },
    {
      "accountBalance": 9300.0,
      "email": "cynthia.gomez@example.org",
      "id": "3087",
      "isActive": true,
      "name": "Cynthia Gomez",
      "profile": {
```

```
            "age": 41,
            "height": 5.4,
            "preferences": {
              "email": true,
              "theme": "auto"
            }
          }
        }
      ]
}
{
  "users": [
    {
      "accountBalance": "2489.75",
      "_9email": "alice.johnson@example.com",
      "id1": "1023",
      "isActive": "true",
      "na_x0026_me": "Alice Johnson",
      "foo": {
        "_x0040_nil": "true"
      },
      "profile": {
        "age": "34",
        "height": "5.7",
        "preferences": {
          "email": "false",
          "theme": "dark"
        }
      }
    }
  ]
}
```

1.2.3 Escape and Mangle of invalid XML chars

1. Input Properties:
   json.dataTypeConvertor.escapeInvalidStart = 'true'
   json.dataTypeConvertor.mangleInvalidChars = 'true'
   json.dataTypeConvertor.escapeChar = '_'

2. Input Payload:

```
{
 "users": [
  {
   "accountBalance": "2489.75",
   "9email": "alice.johnson@example.com",
   "id1": "1023",
   "isActive": "true",
   "na&me": "Alice Johnson",
   "foo": {
    "@nil": "true"
   },
   "profile": {
    "age": "34",
    "height": "5.7",
    "preferences": {
     "email": "false",
     "theme": "dark"
    }
   }
  }
 ]
}
```

3. Output Payload:

```
{
 "users": [
  {
   "accountBalance": "2489.75",
   "_9email": "alice.johnson@example.com",
   "id1": "1023",
   "isActive": "true",
   "na_u0026me": "Alice Johnson",
   "foo": {
    "_u0040nil": "true"
   },
   "profile": {
    "age": "34",
    "height": "5.7",
    "preferences": {
     "email": "false",
     "theme": "dark"
    }
   }
  }
 ]
}
```